

1. resumo do problema em linguagem própria;
O problema se trata de logística para maximização de pares de dança em uma determinada festa escolar, onde terá a quantidade de garotos e garotas, além de suas preferências para parceiros de dança.
2. interpretação da entrada e da saída;
Na primeira linha são 3 inteiros, 'n', 'm' e 'k'; onde 'n' é o número de garotos, 'm' o número de garotas e 'k' a quantidade de potenciais pares.
Em seguida as k seguintes são compostas de inteiros 'a' e 'b'; onde a é o index do garoto e 'b' o index da garota, esses pares indicam que dançariam juntos
3. modelagem da rede de fluxo;
um grafo de fluxo, onde a entrada vai sair todos os pares possíveis ligando com os vértices dos garotos que serão ligados com as possíveis parceiras e dessas sairiam para o final 'r', caso o fluxo seja maximizado, 'r' recebera o maximo de possíveis pares
4. justificativa da escolha entre Ford-Fulkerson e Edmonds-Karp;
foi escolhido o edmonds-karp pois ele utiliza BFS para encontrar caminhos aumentantes. Como todas as capacidades da rede são unitárias, o algoritmo encontra o fluxo máximo de maneira eficiente.

5. instância pequena;

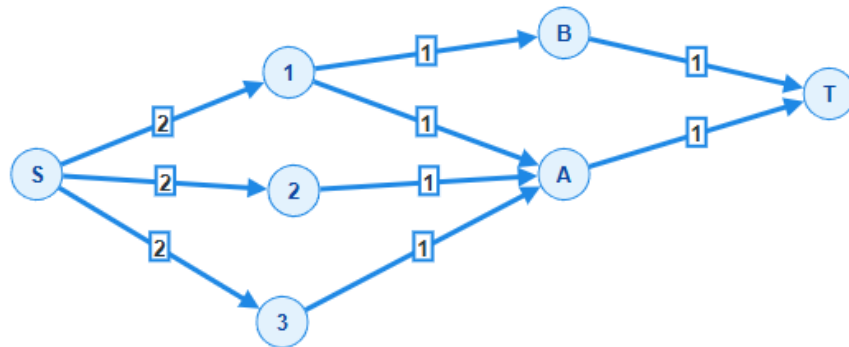
Entrada:

3 2 4

1 1

1 2
2 1
3 1

Grafo gerado:



6. execução manual passo a passo;

Mapeamento dos vértices para índices numéricos do algoritmo:

- * S (Fonte) = 0
- * 1 = 1
- * 2 = 2
- * 3 = 3
- * A = 4
- * B = 5
- * T (Sorvedouro) = 6

Estado Inicial

- * flows: [0->1 0.00/1.00; 0->2 0.00/1.00; 0->3 0.00/1.00; 1->4 0.00/1.00; 1->5 0.00/1.00; 2->4 0.00/1.00; 3->4 0.00/1.00; 4->6 0.00/1.00; 5->6 0.00/1.00]
- * value: 0.00

Passo 0: Aumentar ao longo de 0 -> 1 -> 4 -> 6

A BFS começa em 0. Ao varrer as adjacências, visita os nós 1, 2 e 3. A partir de 1, alcança 4 e 5. A partir de 4, alcança o sorvedouro 6.

- * marked[]: [0:T; 1:T; 2:T; 3:T; 4:T; 5:T; 6:T]
- * edgeTo[]: 1:0->1 0.00/1.00; 2:0->2 0.00/1.00; 3:0->3 0.00/1.00; 4:1->4 0.00/1.00; 5:1->5 0.00/1.00; 6:4->6 0.00/1.00
- * bottleneck: `1.00`
- * Mudanças de fluxo: adiciona 1.00 em 4->6, 1->4 e 0->1.

* flows: = [0->1 1.00/1.00; 0->2 0.00/1.00; 0->3 0.00/1.00; 1->4 1.00/1.00; 1->5 0.00/1.00; 2->4 0.00/1.00; 3->4 0.00/1.00; 4->6 1.00/1.00; 5->6 0.00/1.00]
* value: 1.00

Passo 1: Aumentar ao longo de 0 -> 2 -> 4 -> 1 -> 5 -> 6 *Nota de execução residual:

A aresta direta 0->1 está cheia. A BFS inicia em 0, visita 2 e 3. A partir de 2, alcança 4 (via 2->4). Como 4->6 está cheia, a BFS usa a aresta reversa de fluxo de 1->4 para alcançar o nó 1 de forma residual, permitindo acessar 1->5 e finalmente chegar a 6 via 5->6.

* marked[]: [0:T; 1:T; 2:T; 3:T; 4:T; 5:T; 6:T]
* edgeTo[]: 2:0->2 0.00/1.00; 3:0->3 0.00/1.00; 4:2->4 0.00/1.00; 1:4->1 1.00/1.00 (reversa); 5:1->5 0.00/1.00; 6:5->6 0.00/1.00
* bottleneck: 1.00
* Mudanças de fluxo: adiciona 1.00 em 5->6, 1->5 e 2->4; subtrai 1.00 (desfaz) em 1->4.
* flows: [0->1 1.00/1.00; 0->2 1.00/1.00; 0->3 0.00/1.00; 1->4 0.00/1.00; 1->5 1.00/1.00; 2->4 1.00/1.00; 3->4 0.00/1.00; 4->6 1.00/1.00; 5->6 1.00/1.00]
* value: 2.00

BFS Final: Não resta caminho aumentante A BFS começa em 0. Alcança o nó 3 pela capacidade residual restante em 0->3. A partir de 3, alcança o nó 4 via 3->4. A partir de 4, todas as conexões diretas para frente (4->6) estão cheias, e o caminho de retorno para 1 (1->4) agora possui fluxo zero, impedindo progressão residual. Os nós 5 e 6 não podem ser alcançados.

* marked[]: [0:T; 1:F; 2:F; 3:T; 4:T; 5:F; 6:F]
* Lado da fonte do mincut: {0, 3, 4}
* Valor final do fluxo máximo: 2.00

7. verificação da resposta final.

